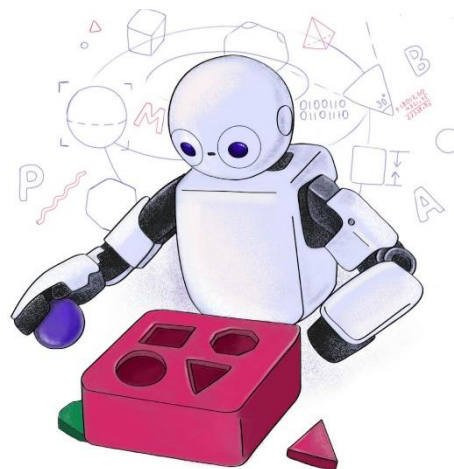


C24 – Inteligência Artificial: *Redes neurais recorrentes*



Motivação

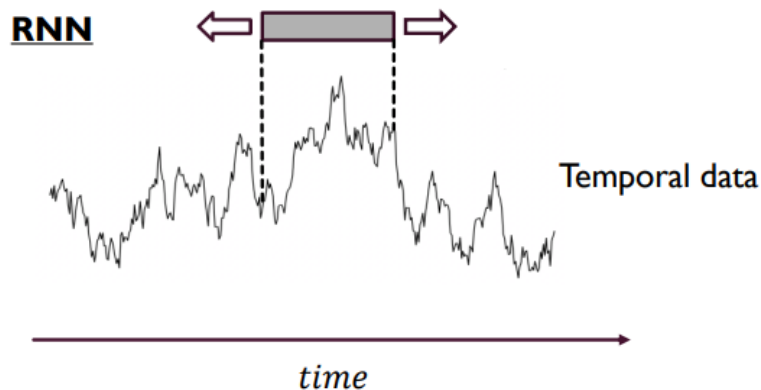
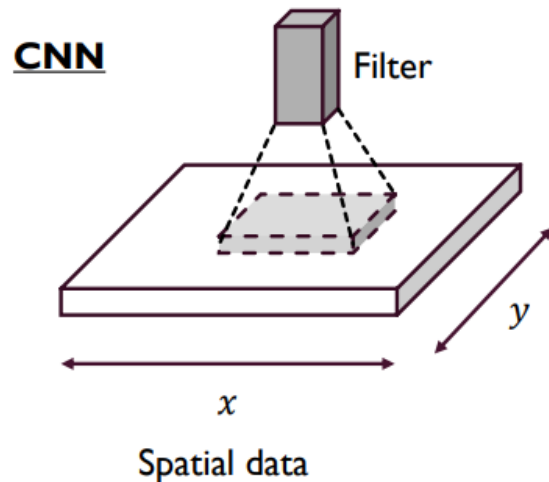
- Muitos problemas envolvem **dados que são sequenciais**:
 - Voz
 - Texto
 - Séries temporais (e.g., EGC, EEG, preço de ações, desvanecimento de um canal)
 - Vídeo
- Para lidarmos com eles, precisamos **modelar dependências temporais!**
- Em outras palavras, existe **correlação temporal entre as amostras**.

Cada amostra carrega informação das anteriores.

Aplicações

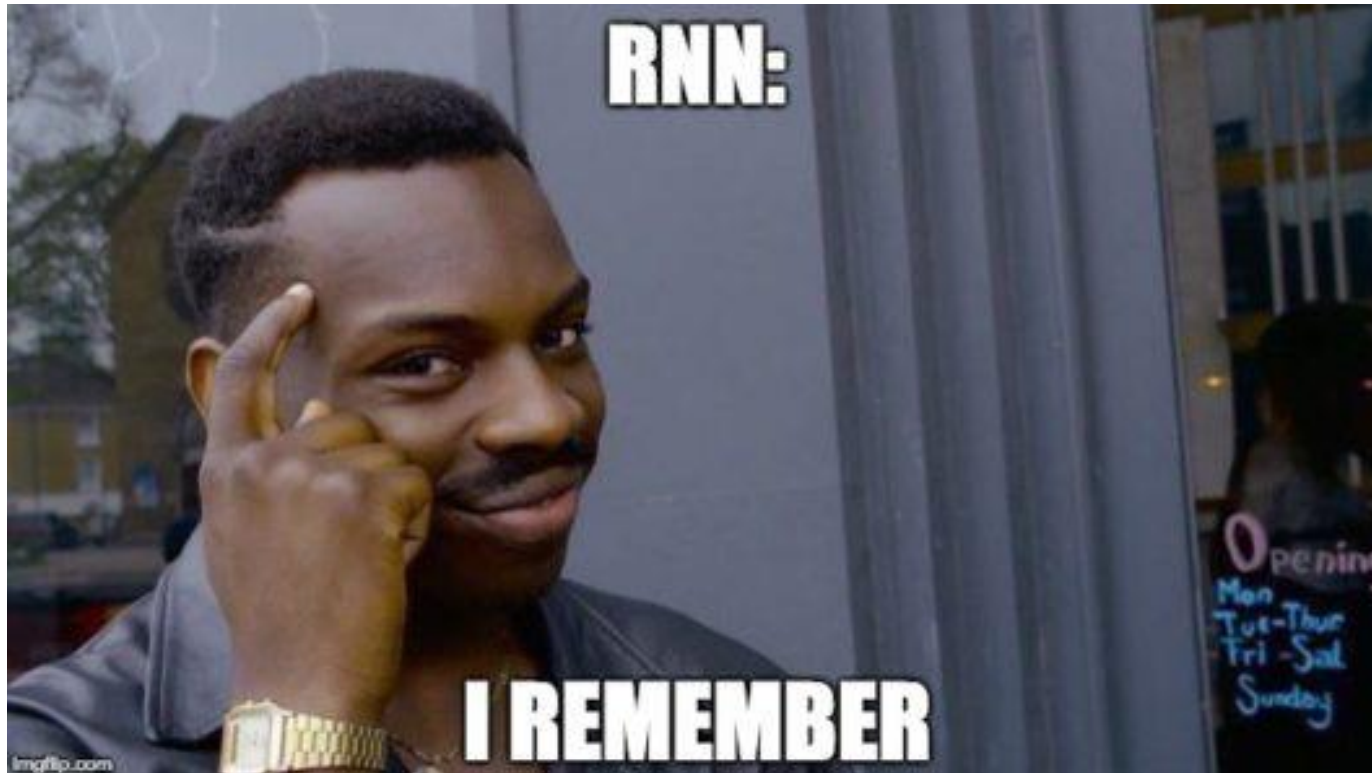
- Previsão de séries temporais (e.g., clima, ações, vendas, etc.)
- Tradução automática de texto
- Reconhecimento de fala
- Tradução fala para texto e vice versa
- Classificação de sentimentos
- Geração de texto
- Geração de descrições de texto para imagens (*image captioning*)
- Rastreamento de objetos

Limitação de redes tradicionais



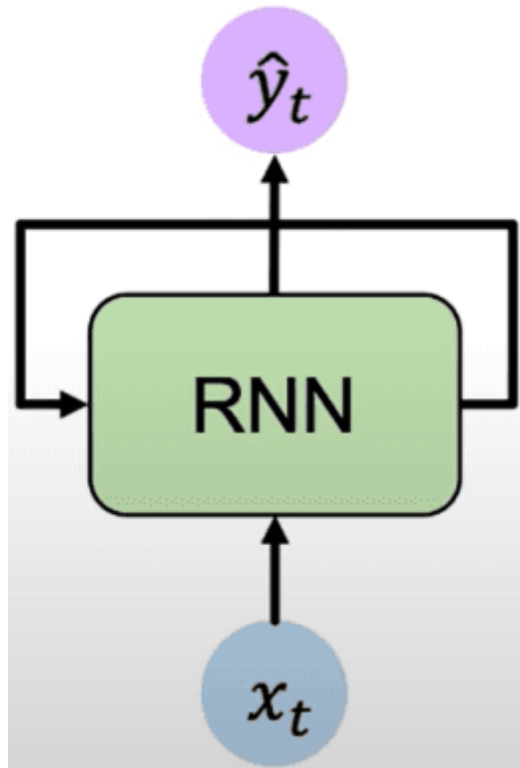
- MLPs:
 - Não armazenam informações de estados anteriores (i.e., sem memória)
 - Cada entrada é tratada de forma independente.
 - Não capturam dependências temporais.
 - Por ser de alimentação direta, a saída depende apenas da entrada atual.
- CNNs:
 - Compartilham as limitações das MLPs, mas com uma exceção: elas modelam relações espaciais.
- Ambas famílias exigem que o vetor de entrada tenha um tamanho predefinido e constante.

Como podemos considerar as correlações temporais e trabalhar com entradas sem tamanho definido?



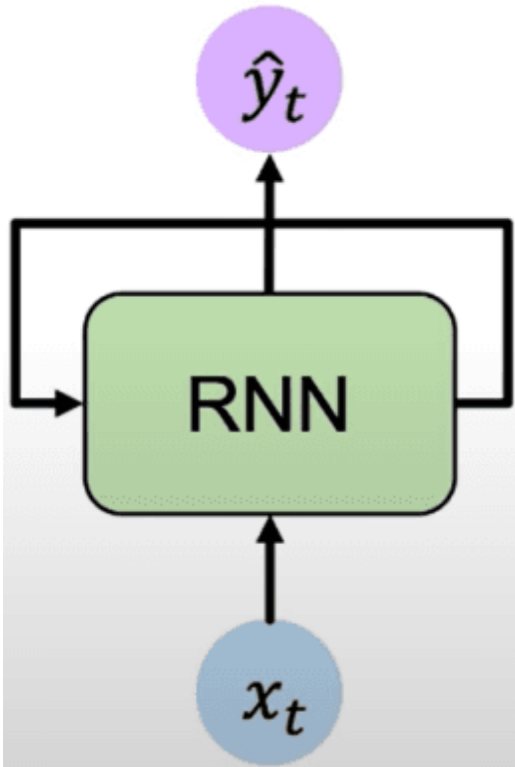
Redes neurais
recorrentes
têm memória!

Redes recorrentes



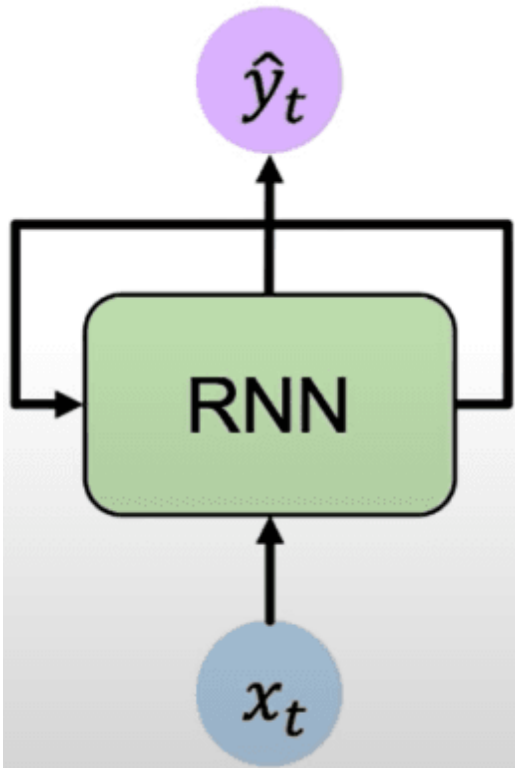
- Possuem **laços de realimentação**.
- Mantém um **estado interno** (**memória**) que **armazena informações** observadas até o momento.
- A **saída** da rede depende da **entrada atual** e do valor do **estado interno anterior**.
- O **estado interno** mantém uma **memória** do histórico de **todas as entradas passadas**.
- A **memória desaparece** ao longo do tempo.

Redes recorrentes



- Uma RNN utiliza **informações sequenciais**, **modelando as dependências temporais** entre as amostras de entrada.
- **Exemplo:** se quisermos **prever a próxima palavra** em uma frase, precisamos saber **quais palavras vieram antes dela**.

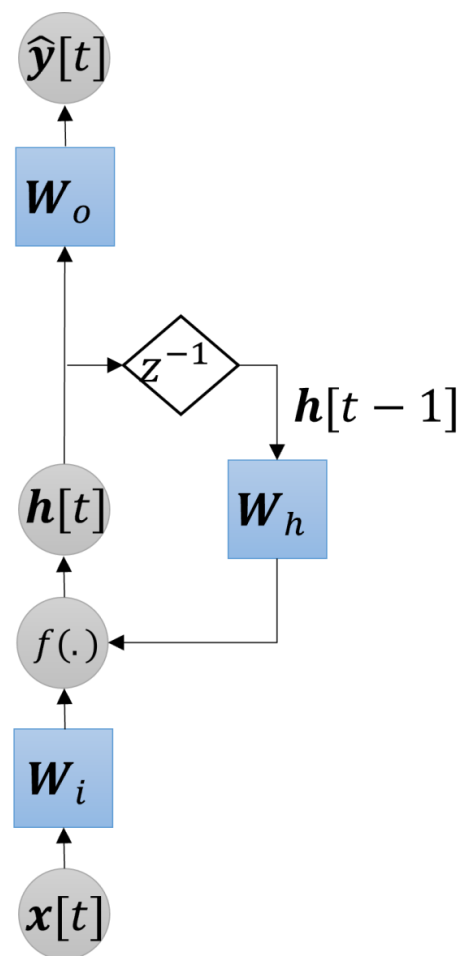
Redes recorrentes



- RNNs são projetadas para processar sequências de **comprimento variável**.
- Elas usam o **histórico de entradas anteriores** para **gerar uma saída**.
- Por exemplo, na frase:
“O gato subiu no _____”
a rede usa as palavras anteriores para prever “telhado”.

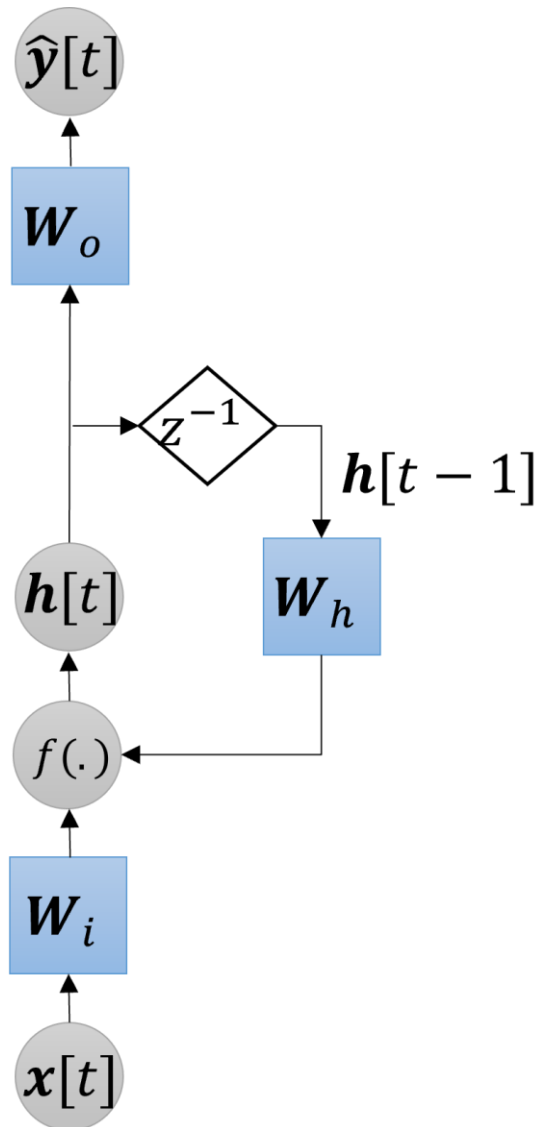
O que compõem as RNNs?

Neurônio (ou célula) recorrente



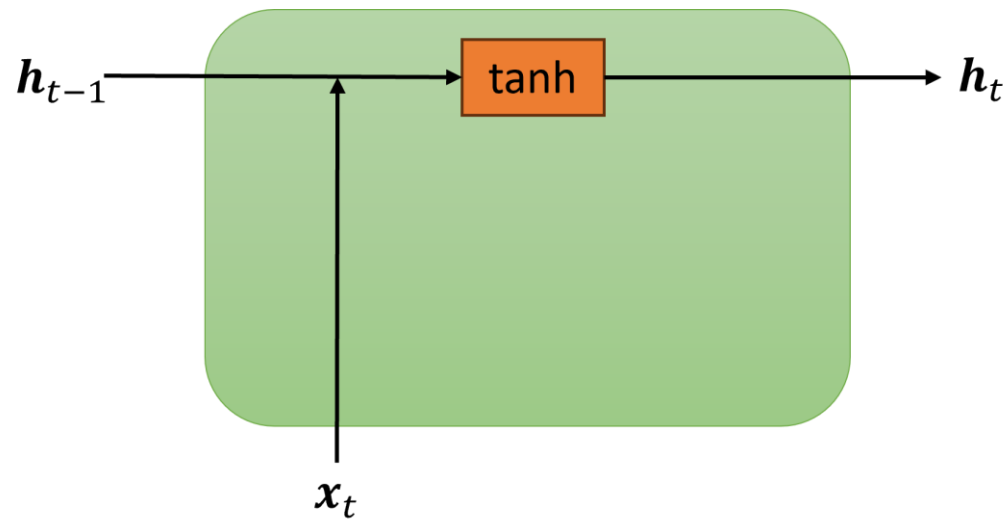
As RNNs são compostas por **neurônios recorrentes**.

Neurônio (ou célula) recorrente



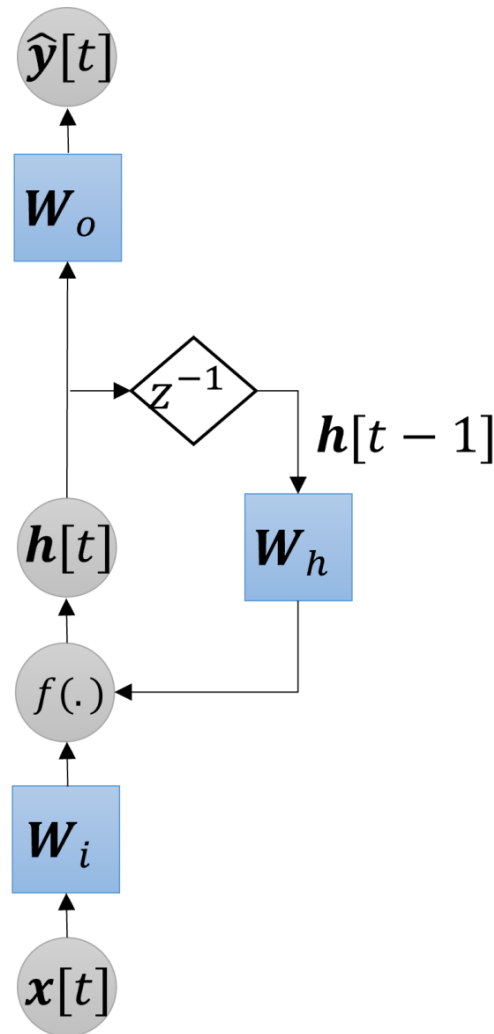
- $x[t]$: entrada
- $\hat{y}[t]$: saída
- $h[t]$: estado interno ou oculto (**memória da rede**)
- $h[t - 1]$: estado interno anterior
- W_i : pesos de entrada
- W_h : pesos da camada recorrente (oculta)
- W_o : pesos de saída
- z^{-1} : unidade de retardo de tempo
- $\tanh$: função de ativação, $f(\cdot)$

Neurônio recorrente



- Representação **simplificada** do neurônio.
- Essa representação é bastante utilizada na literatura.

Atualização do estado e geração da saída



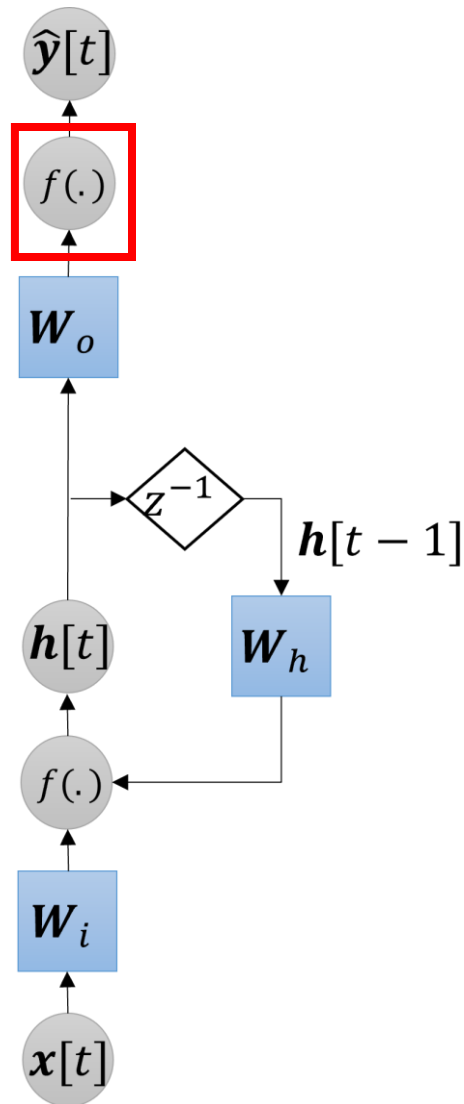
- Equações de **atualização do estado oculto** e de **saída do neurônio** recorrente:

$$h[t] = f(W_h h[t-1] + W_i x[t]),$$

$$\hat{y}[t] = W_o h[t].$$

- Os **pesos são os mesmos** em **todos os intervalos de tempo**.
- Em geral, usa-se funções de ativação, $f(\cdot)$, do tipo **tangente hiperbólica** (tanh) ou **sigmóide**.
 - Não se usa ReLU, pois **por não saturarem**, a **recorrência** pode **causar explosão do gradiente**.

Atualização do estado e geração da saída

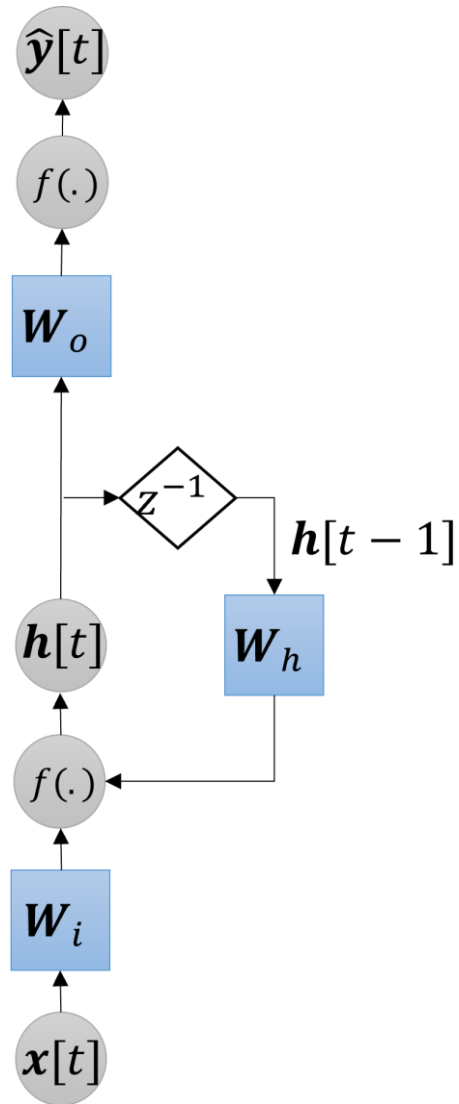


- Algumas implementações consideram uma função de ativação na saída.
- Equação de **saída do neurônio** recorrente fica assim:

$$\hat{y}[t] = f(W_o h[t]).$$

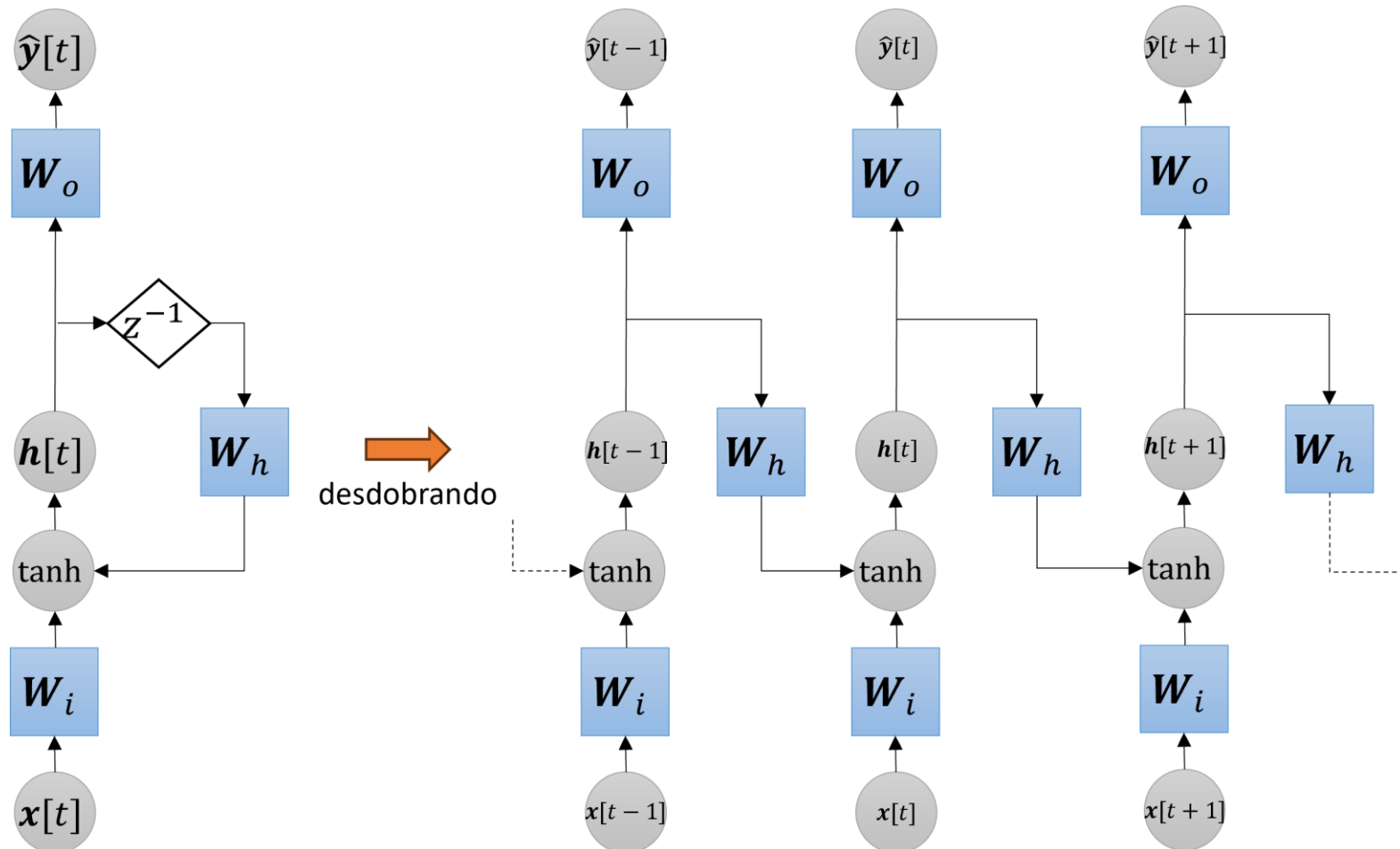
- A equação de **atualização do estado oculto** permanece a mesma.

Atualização do estado e geração da saída



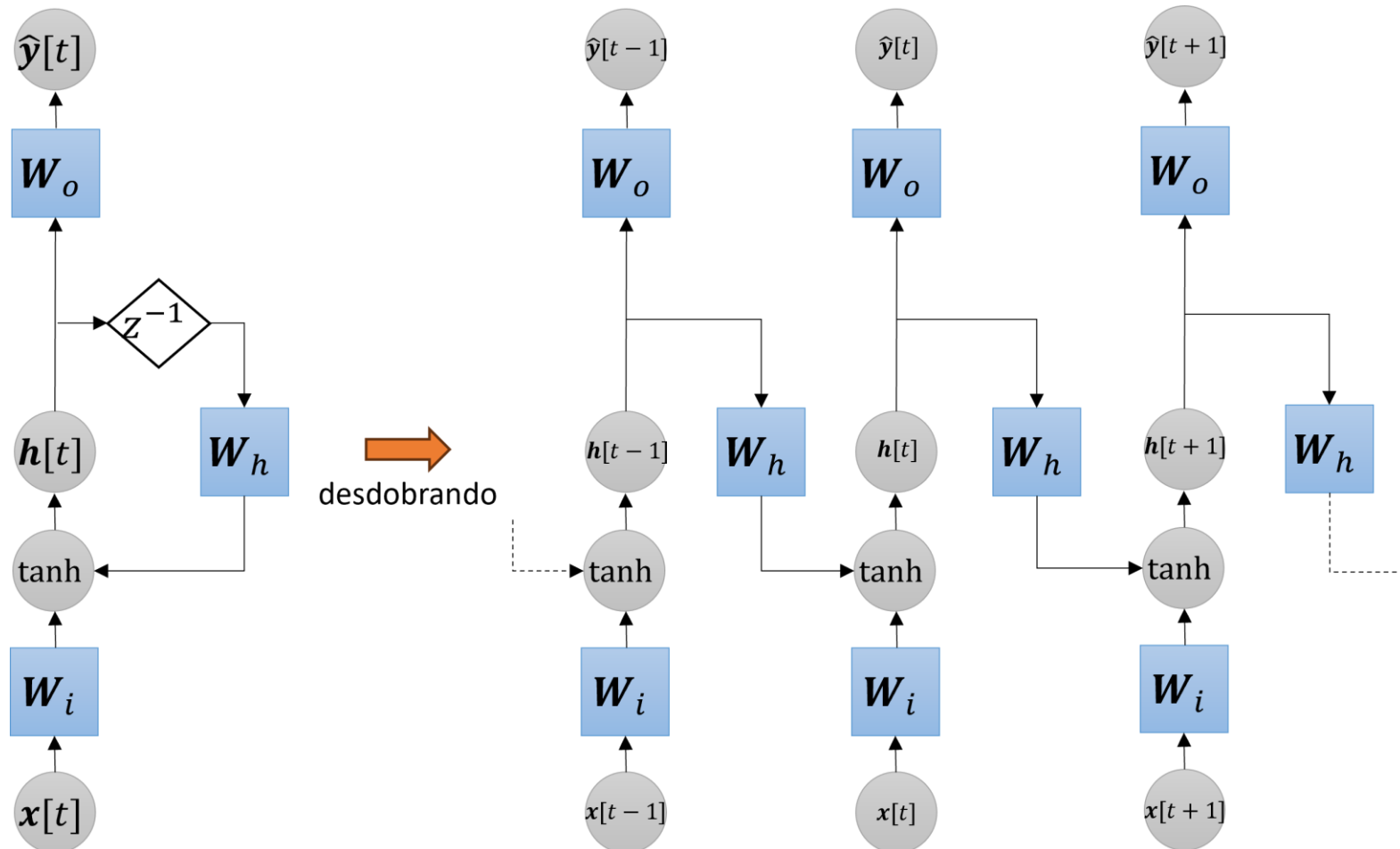
- Podemos **desdobrar** uma RNN para entender como a **memória evolui ao longo do tempo**.
- Isso nos ajuda a
 - entender como ela processa sequências,
 - e a treinar a rede usando *backpropagation*;

Desdobrando uma RNN



- As RNNs podem ser vistas como **múltiplas cópias da mesma rede**, cada uma passando **uma mensagem para a sua sucessora**.
- É como se tivéssemos uma rede MLP com várias camadas.

Desdobrando uma RNN

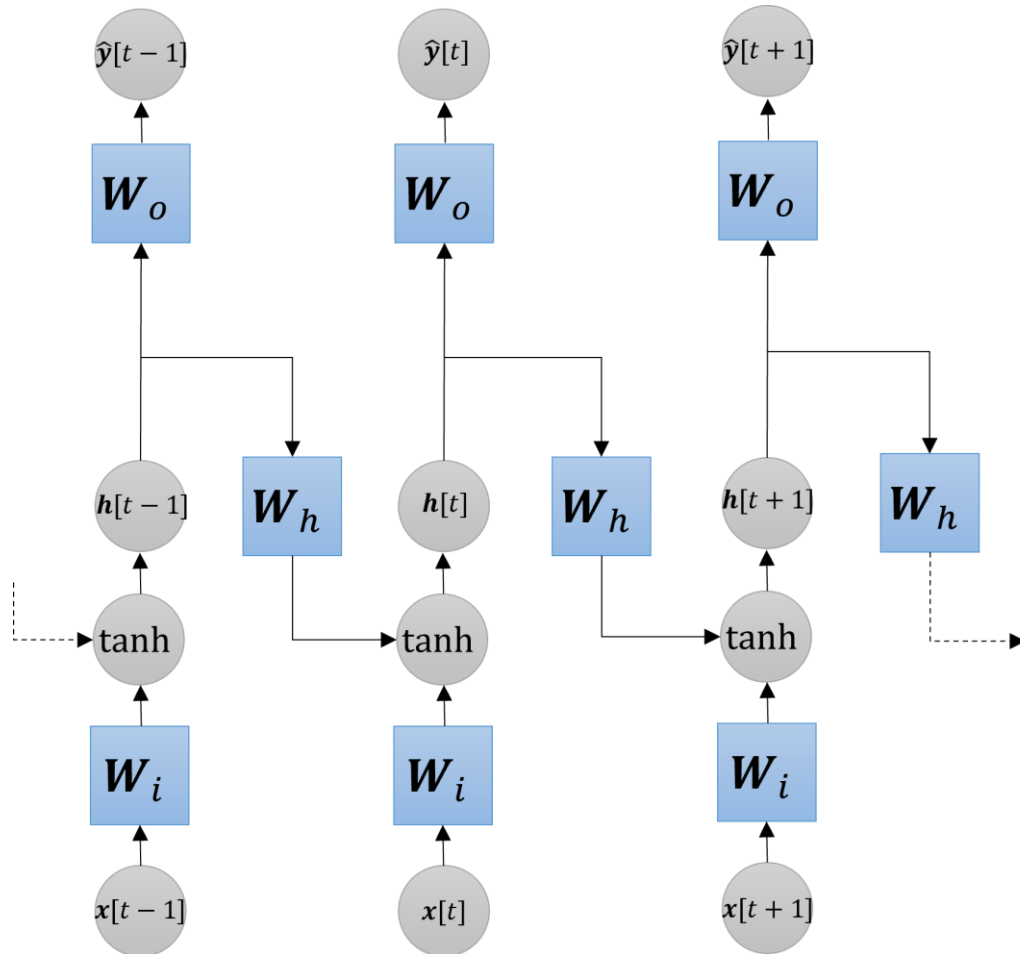


- A **mesma função de ativação** e o mesmo **conjunto de pesos** são usados **em cada passo** de tempo.
- Percebam que uma saída $\hat{y}[t]$ depende dos valores anteriores e não apenas de $x[t]$ como em uma MLP.

Treinando uma RNN

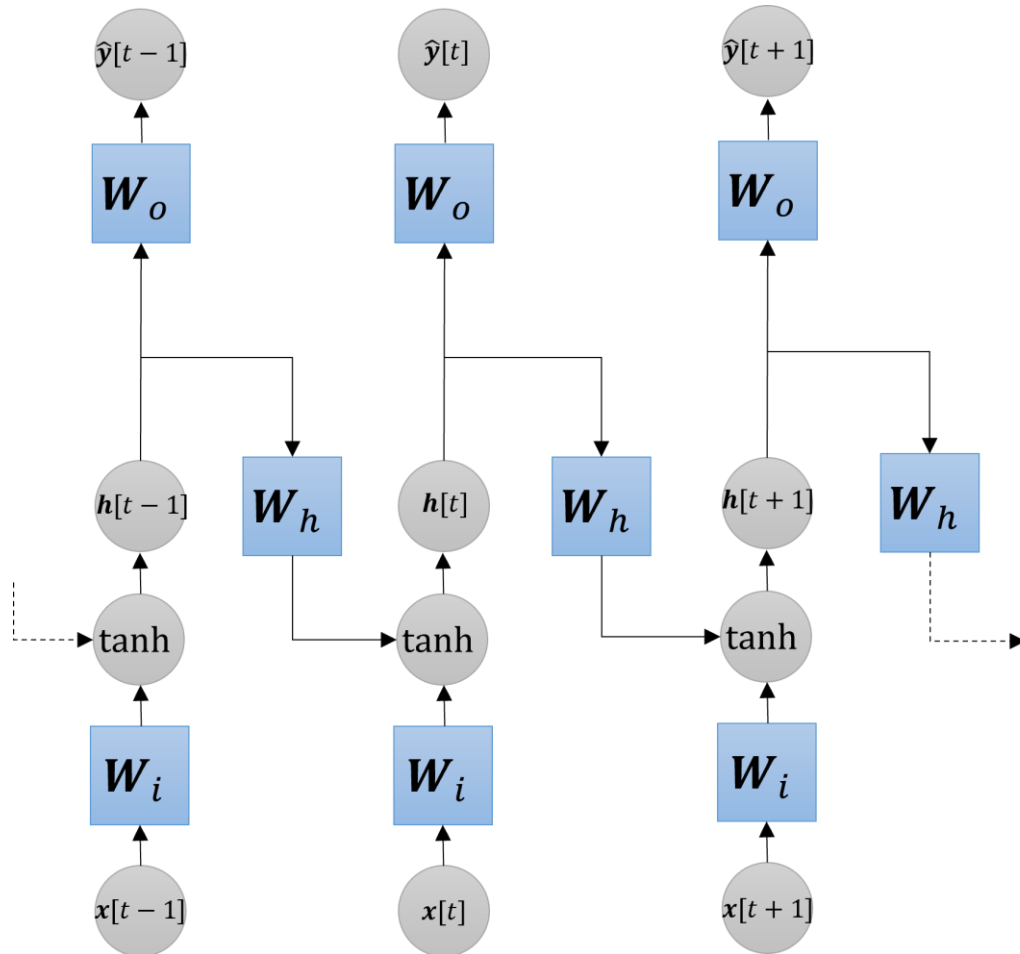
- Os pesos são atualizados usando o **gradiente descendente**.
- Uma **função de erro** (EQM, EC, etc.) é usada para calcular o erro entre as saídas da rede e os rótulos do conjunto de treinamento.
- A **derivada da função de erro**, em relação aos pesos da rede, é **retropropagada** através da rede.
- Os pesos são ajustados até que um critério de parada seja atendido:
 - O número máximo de épocas é atingido.
 - A função de perda para de diminuir.

Treinando uma RNN



- Para treinar a rede, ela é primeiro **desdobrada** e depois aplica-se a **retropropagação** do erro **tradicional**.
- Cada **réplica** da rede é **relativa** a um **intervalo de tempo diferente**.
- Cada **réplica** pode ser pensada como **uma camada em uma rede MLP**.
- Percebam que a **arquitetura da rede** pode se tornar **muito profunda**.
- Os **pesos são mantidos constantes** em cada **intervalo de tempo**.

Treinando uma RNN



- Esse desdobramento da rede para se aplicar a retropropagação do erro é chamado de *Backpropagation Through Time* (BPTT).
- Ele é uma **extensão** do *backpropagation* tradicional, mas **aplicado ao longo do tempo**.
- A ideia central é:
o **erro atual** pode ter **sido causado por estados passados** da rede.
- Então o gradiente precisa “voltar no tempo”.

Backpropagation Through Time (BPTT)

- A diferença entre a saída da rede e o valor desejado é avaliada por uma função de erro **ao longo de todos os t passos de tempo**

$$E_t = \frac{1}{t} \sum_{i=0}^t e(y[i], \hat{y}[i]) .$$

- Em outras palavras é uma **media temporal dos erros** cometidos pela rede **ao longo de todos os instantes de tempo, t .**

Backpropagation Through Time (BPTT)

- Por exemplo, usando a retropropagação (regra da cadeia) para encontrar os gradientes com relação aos pesos $\mathbf{W}_h$, temos

$$\frac{\partial E_t}{\partial \mathbf{W}_h} = \frac{1}{t} \sum_{i=0}^t \frac{\partial e(y[i], \hat{y}[i])}{\partial \mathbf{W}_h} = \frac{1}{t} \sum_{i=0}^t \frac{\partial e(y[t], \hat{y}[t])}{\partial \hat{y}[t]} \frac{\partial \hat{y}[t]}{\partial h[t]} \boxed{\frac{\partial h[t]}{\partial h[i]}} \frac{\partial h[i]}{\partial \mathbf{W}_h}.$$

- Notem que **somamos as contribuições de cada passo de tempo** para obter o gradiente.
- O primeiro, o segundo e o quarto fatores do produto são fáceis de calcular.
- O terceiro é onde as coisas ficam complicadas, já que precisamos **calcular recursivamente** o efeito de $h[i]$ em $h[t]$.

Backpropagation Through Time (BPTT)

$$\frac{\partial E_t}{\partial \mathbf{W}_h} = \frac{1}{t} \sum_{i=0}^t \frac{\partial e(y[t], \hat{y}[t])}{\partial \hat{y}[t]} \frac{\partial \hat{y}[t]}{\partial h[t]} \frac{\partial h[t]}{\partial h[i]} \frac{\partial h[i]}{\partial \mathbf{W}_h}.$$

- Se o valor de t for grande (i.e., uma **sequência muito longa**), temos uma **profundidade intratável**, o que torna o **treinamento muito lento**, **consume muita memória** e pode deixar o **treinamento instável devido à recursão**.
- Na prática, muitas vezes **não propagamos até o início da sequência**, mas até um determinado instante de tempo, τ .
 - Esse processo é chamado de **BPPT truncado**.
- Ele reduz:
 - custo computacional,
 - e uso de memória.

Por que o BPTT truncado funciona?

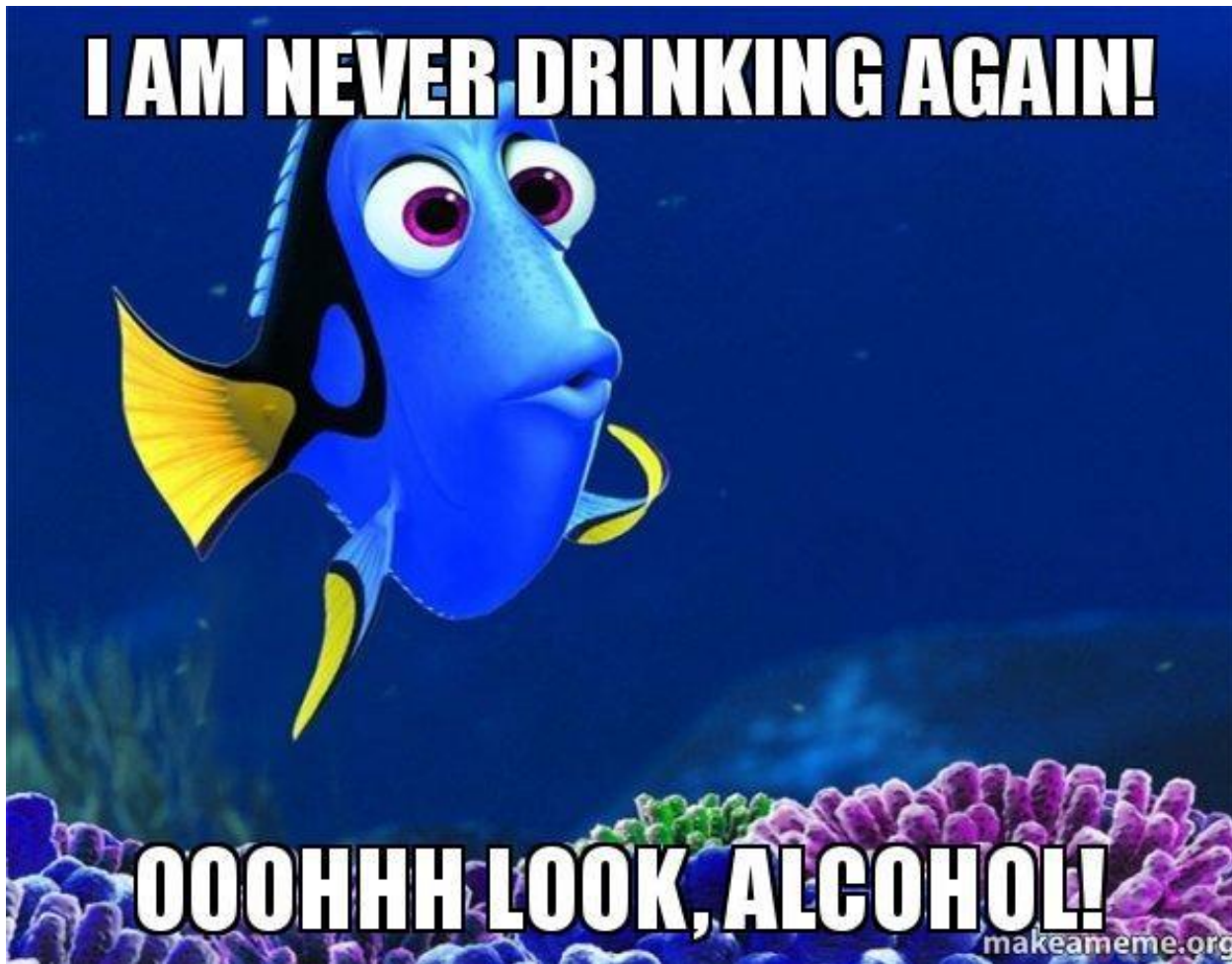
- Funciona porque, na prática, **muitas dependências temporais relevantes são locais**.
- Além disso, muitas **dependências temporais relevantes decaem com o tempo**.
- Isso **evita** a necessidade de **voltar até o primeiro passo** da sequência **para calcular o gradiente**.
- Ou seja:
 - o **estado atual** geralmente depende mais do **passado recente**,
 - **contribuições** muito **antigas** tendem a **ficar pequenas**.

Desaparecimento e explosão do gradiente

- Durante o treinamento, o BPTT aplica a regra da cadeia ao longo de vários passos no tempo

$$\frac{\partial E_t}{\partial \mathbf{W}_h} = \frac{1}{t} \sum_{i=0}^t \frac{\partial e(y[t], \hat{y}[t])}{\partial \hat{y}[t]} \frac{\partial \hat{y}[t]}{\partial h[t]} \left(\prod_{k=i}^{t-1} \frac{\partial h[k+1]}{\partial h[k]} \right) \frac{\partial h[i]}{\partial \mathbf{W}_h}.$$

- O gradiente envolve um **produto de muitas derivadas sucessivas**.
- O **produtório** na equação acima **pode causar o desaparecimento ou a explosão do gradiente**.



RNNs **tradicionais** não
aprendem
dependências longas

Desaparecimento e explosão do gradiente

- Se $\frac{\partial h[k+1]}{\partial h[k]} \forall k < 1$, então o produto tende a **zero**
 - A rede **esquece o passado mais distante** (informação antiga se perde).
 - Não aprende **dependências de longo prazo, apenas curtas**.
 - Estados ocultos anteriores não têm efeito sobre os estados posteriores.
 - Treinamento **muito lento ou estagnado**.
- Se $\frac{\partial h[k+1]}{\partial h[k]} \forall k > 1$, então o produto **explode**
 - Modelo **diverge**.
 - Atualizações de pesos se tornam enormes.
 - **Treinamento instável**: erro vira NaN.
 - Sintoma típico: **oscilação do erro**.
- Na prática, o desaparecimento do gradiente é mais comum.

Como mitigar esses problemas?

- **Para a explosão:**

- Limitar a magnitude do gradiente (*gradient clipping*)
- Inicializar os pesos de forma apropriada (inicializações de He e Xavier)

- **Para o desaparecimento:**

- Usar arquiteturas de RNNs mais avançadas: LSTM e GRU
 - Introduzem o **conceito de *gates***, funcionando como **válvulas de informação e gradiente**
 - ✓ Eles são funções (em geral, sigmoide) que produzem valores entre 0 e 1:
 - 0 → bloqueia informação
 - 1 → deixa passar totalmente
 - Criam caminhos com derivada ≈ 1
 - ✓ **Mantêm gradiente estável**
- **Conexões residuais:** criam **caminhos diretos** para o gradiente
- Inicializar os pesos de forma apropriada (inicializações de He e Xavier)

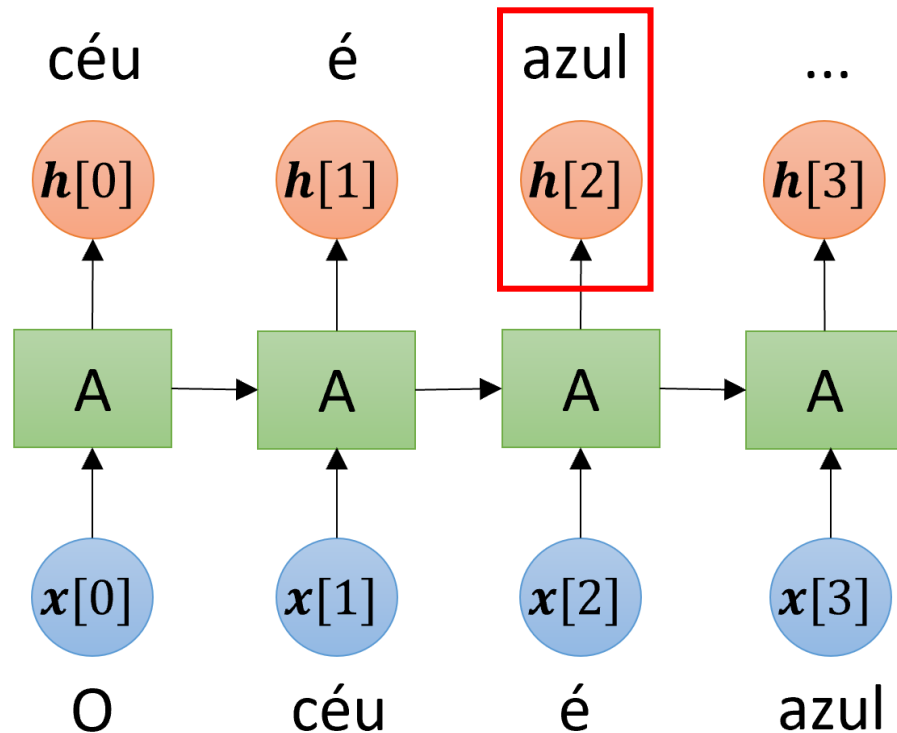
Por que não usar funções de ativação ReLU?

- RNNs **reusam** a **mesma matriz de pesos e a mesma função de ativação repetidamente** em cada passo.
- **Valores maiores que 1** na entrada de uma **função que não satura** como a ReLU ($y = \max(x, 0)$) podem causar um crescimento exponencial (explosão) dos valores do estado oculto, $h[t]$.
- Isso torna o treinamento instável:
 - Os estados podem crescer sem limite, causando dinâmica instável no tempo.
 - Em sistemas dinâmicos (como em RNNs), isso é ruim, pois causa oscilações ou divergência.
- Além disso, **valores negativos** em sua entrada **geram saídas e derivadas iguais a 0**, fazendo com que o **fluxo de aprendizado morra**.

O problema das dependências de longo prazo

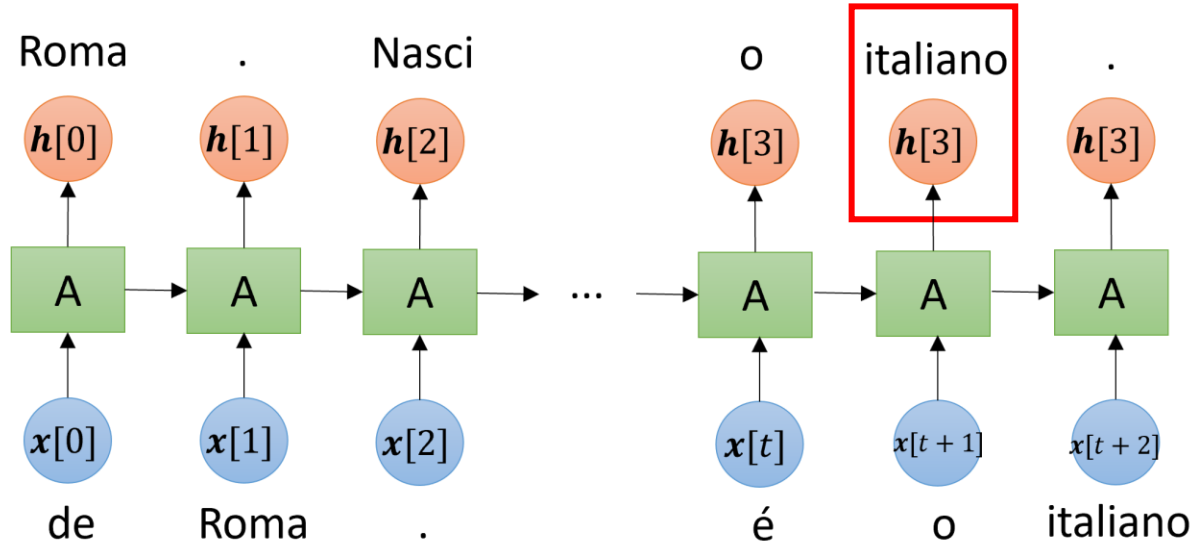
- Devido ao **desaparecimento do gradiente**, as RNNs tradicionais (baunilha) **são incapazes de aprender dependências de longo prazo**.
- Quando uma RNN processa uma **sequência longa**, ela **gradualmente esquece as primeiras entradas** da sequência.
- Entretanto, **algumas aplicações exigem dependências tanto de curto quanto de longo prazo**.
- **Exemplos:**
 - Tradução automática de texto
 - Reconhecimento de fala
 - Rastreamento de objetos

O problema das dependências de longo prazo



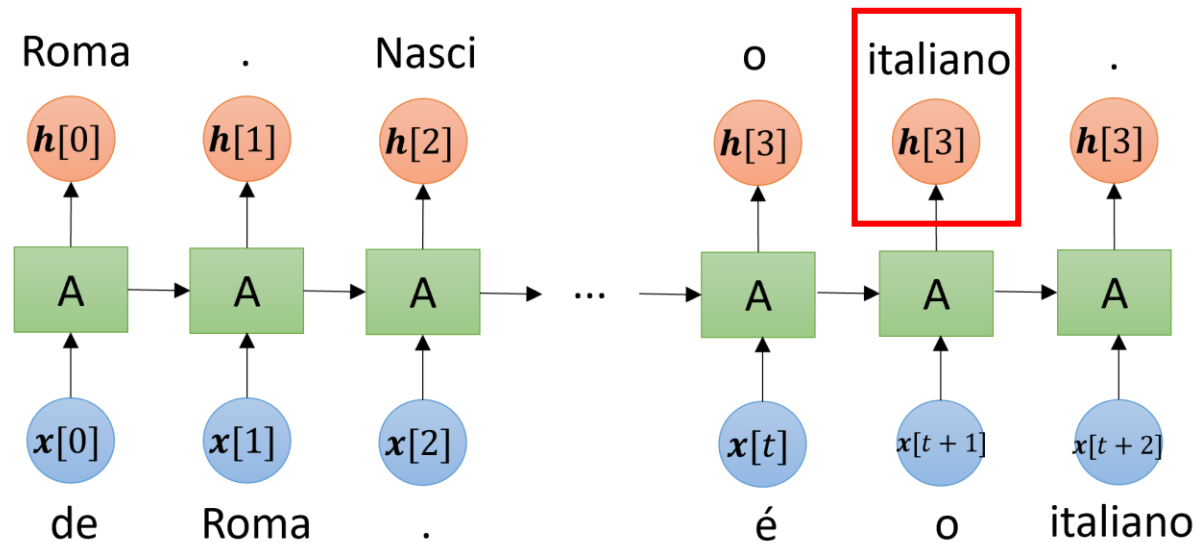
- Considere treinar a RNN ao lado para fazer uma **previsão de um passo à frente**.
- Para **prever a palavra “azul”**, basta olhar apenas **dois passos atrás** no tempo.
- Nesse caso, basta retropropagar o gradiente dois passos para trás para aprender essa tarefa com sucesso.
- Porém, existem casos em que a RNN precisa de mais contexto.

O problema das dependências de longo prazo



- Vamos nos ater à previsão de um passo à frente.
- Considere a frase:
"Sou de Roma. Nasci há 30 anos. Gosto de comer bem e andar de bicicleta. Minha língua materna é o italiano."
- Quando queremos **prever a palavra "italiano"**, temos que olhar **vários passos para trás no tempo**, até a palavra "Roma".

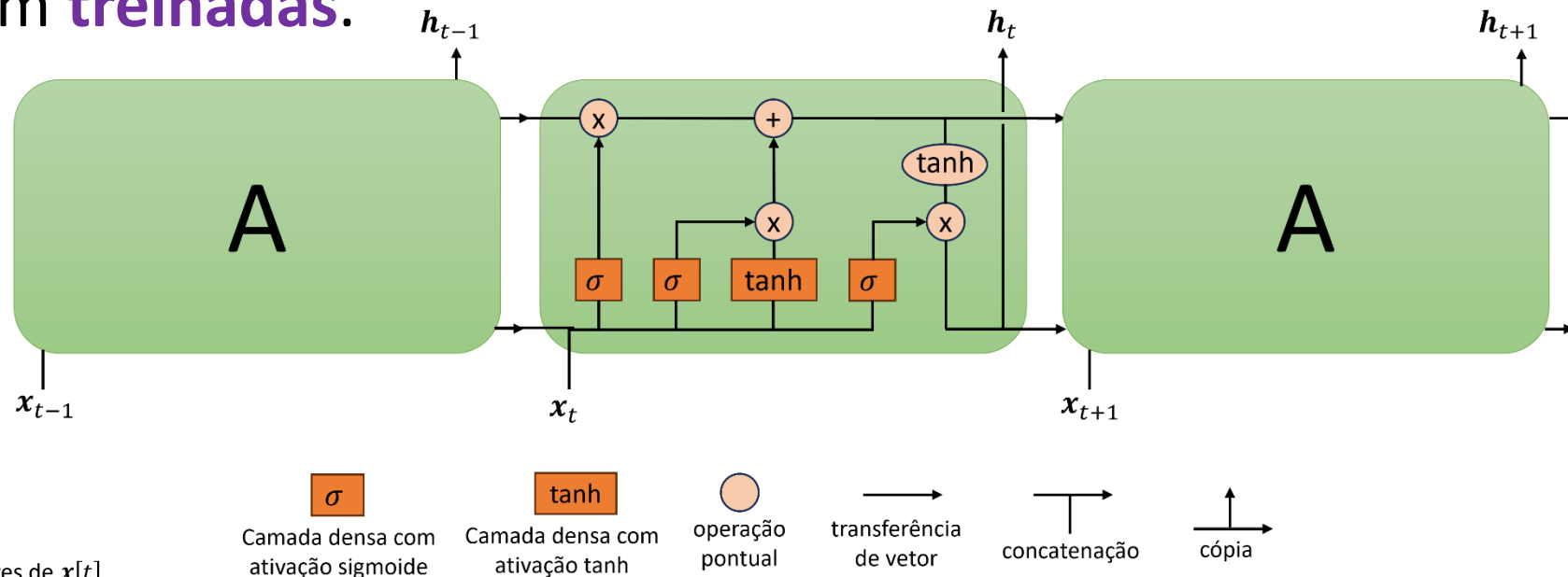
O problema das dependências de longo prazo



- Nesse caso, a memória de curto prazo da RNN tradicional não seria suficiente.
- À medida que o intervalo de tempo aumenta, torna-se mais difícil para uma RNN lidar com o problema.
- Vamos ver como redes de **memória de curto longo prazo (*long-short term memory – LSTM*)** podem lidar com essa dificuldade.

Long-short term memory (LSTM)

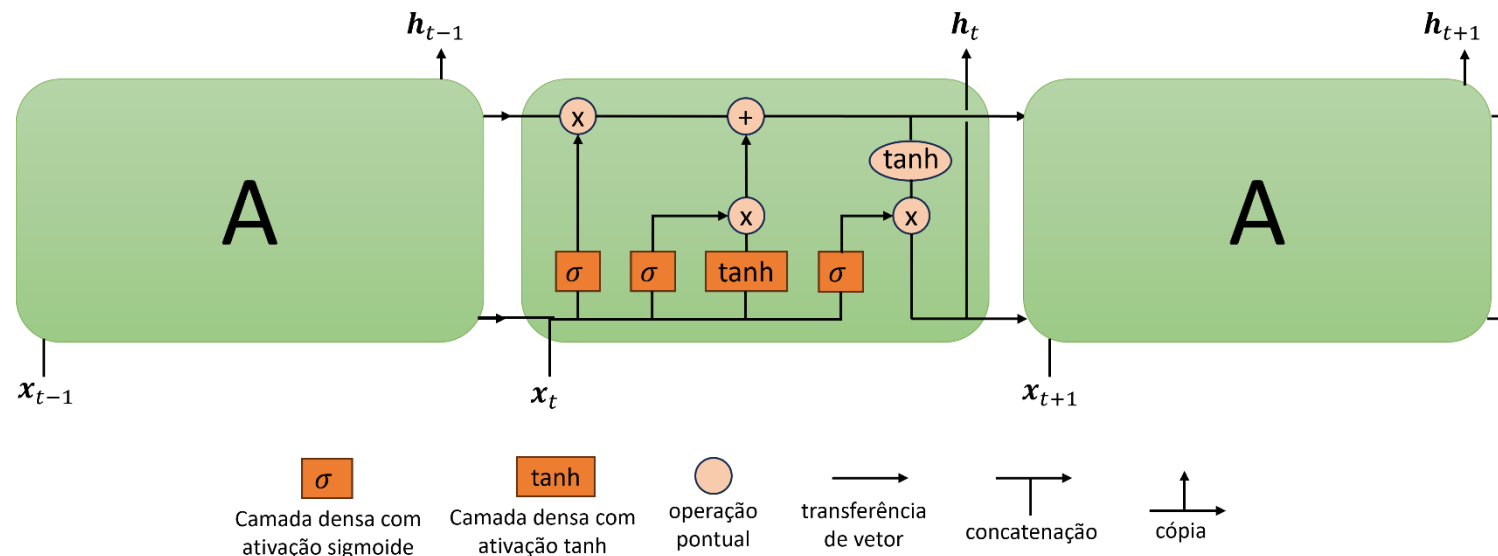
- Introduzida por Hochreiter e Schmidhuber em 1997.
- Funcionam muito bem em **diversos problemas que envolvem dados sequenciais** e são amplamente utilizadas atualmente.
- Assim como as RNNs tradicionais, elas **precisam ser desdobradas** no tempo para serem **treinadas**.



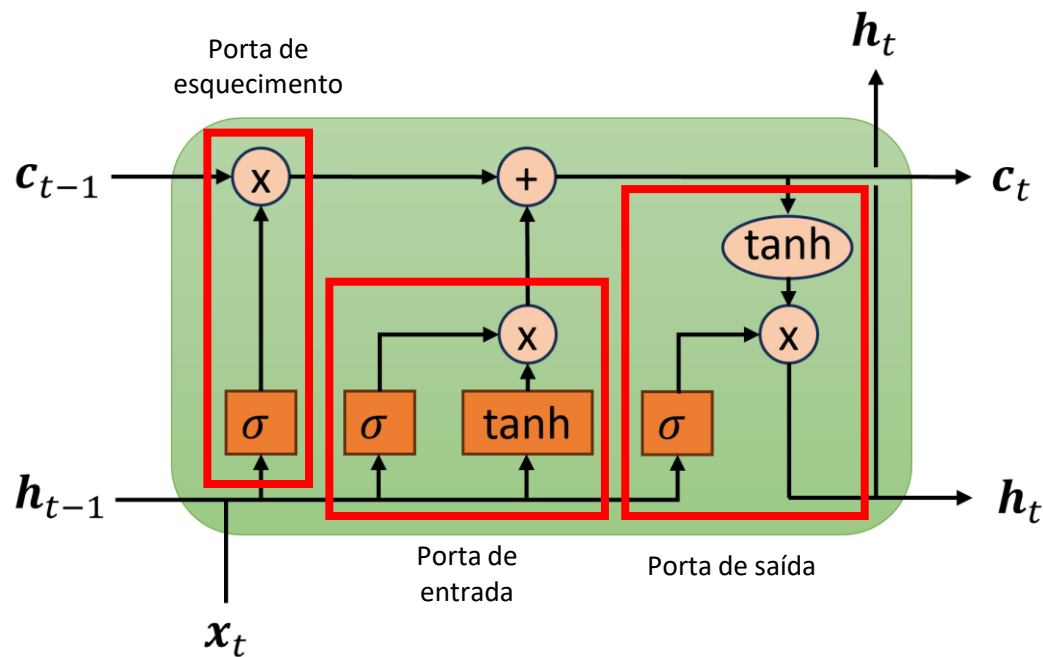
OBS.: Para facilitar a notação, usamos x_t ao invés de $x[t]$.

Long-short term memory (LSTM)

- A **unidade de processamento** (neurônio) da LSTM é mais **complexa** e é chamada de **célula**.
- Uma célula LSTM é composta por **4 camadas totalmente conectadas**, que interagem entre si de uma maneira especial.
- Elas capturam padrões de longo e curto prazo de uma sequência de dados.

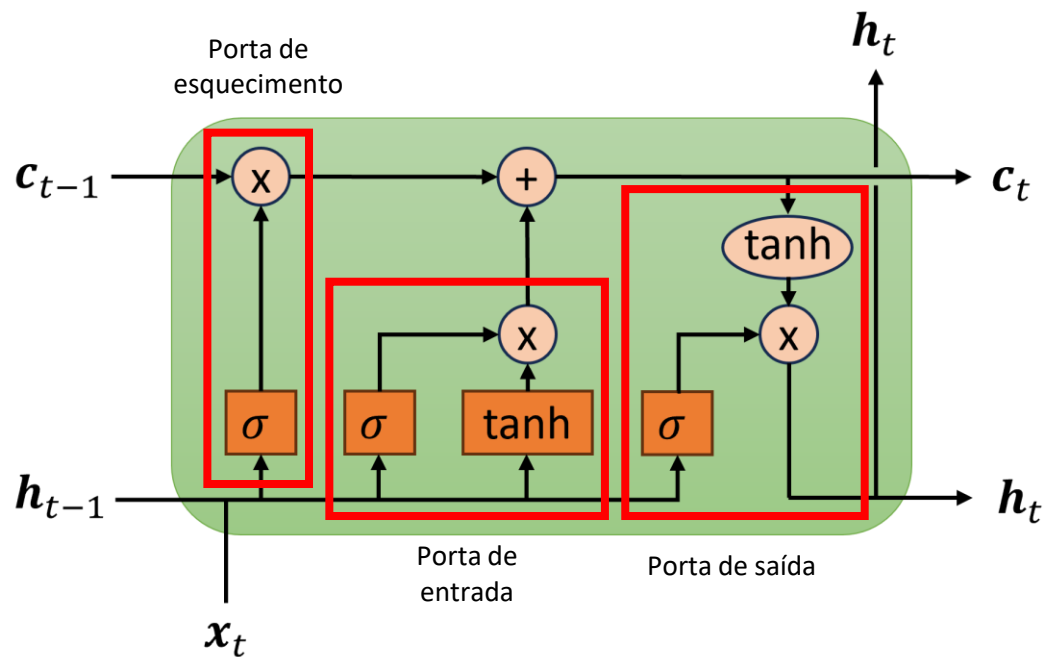


Long-short term memory (LSTM)



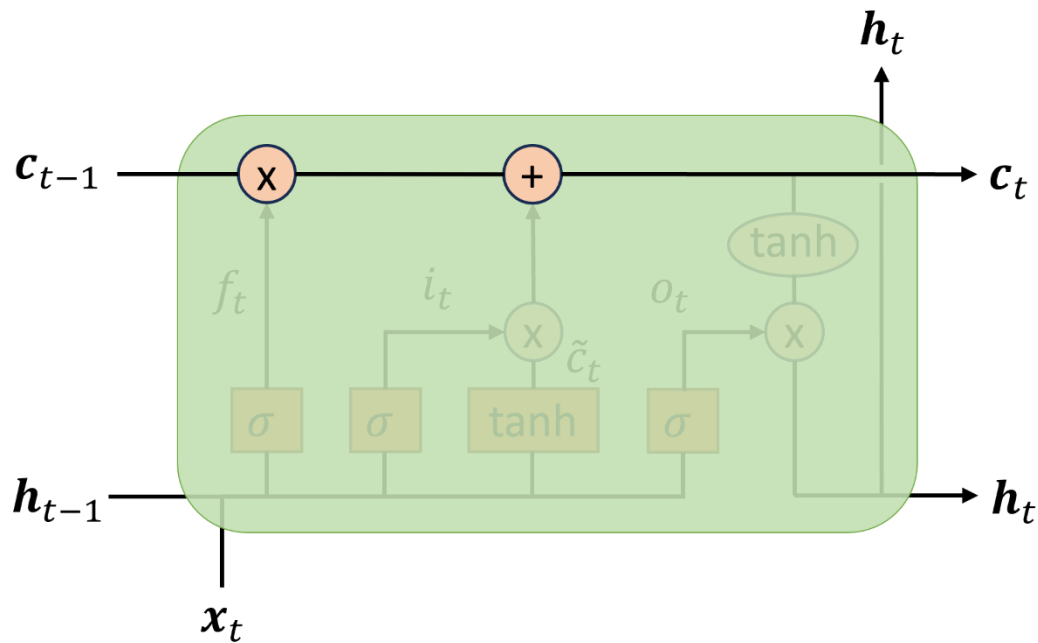
- Uma célula LSTM possui **3 portas**:
 - **Forget gate** (porta de esquecimento)
 - **Input gate** (porta de entrada)
 - **Output gate** (porta de saída)
- A informação na célula é **protegida e controlada** por meio das 3 portas.

Long-short term memory (LSTM)



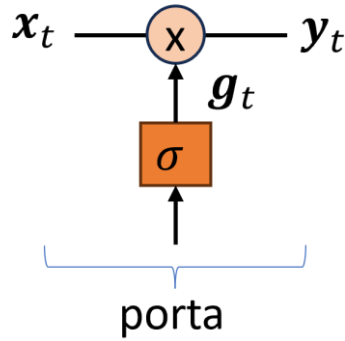
- A **informação entra na célula** sempre que sua **porta de entrada** está **ligada**.
- A **informação é descartada** da célula sempre que sua **porta de esquecimento** está **desligada**.
- A **informação pode ser lida** da célula **ligando sua porta de saída**.

A ideia central por trás das LSTMs: estado da célula



- A **informação** pode **fluir ao longo da célula** de memória **sem sofrer alterações**.
- A **informação** pode ser **removida** ou **gravada** na célula de memória, conforme **regulado pelas portas**.

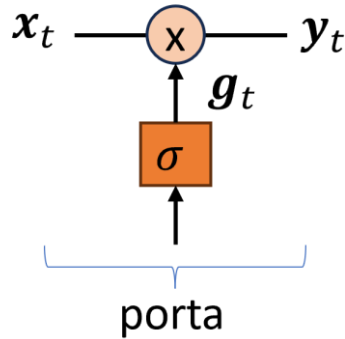
A ideia central por trás das LSTMs: estado da célula



x_t		g_t		y_t
0.58		0.2		0.116
-0.32	$\times$	0.99		-0.3168
0.01		0.1		0.001
1.24		0.95		1.178

- As portas são uma forma de **permitir opcionalmente** a **passagem de informações**.
- Por exemplo, uma camada densa com função de ativação sigmoide gera valores entre 0 e 1.
- Uma **operação de multiplicação elemento a elemento** decide o quanto da entrada passar.
- Com isso a porta decide (**aprende**) o quanto de cada elemento deve ser permitido passar.

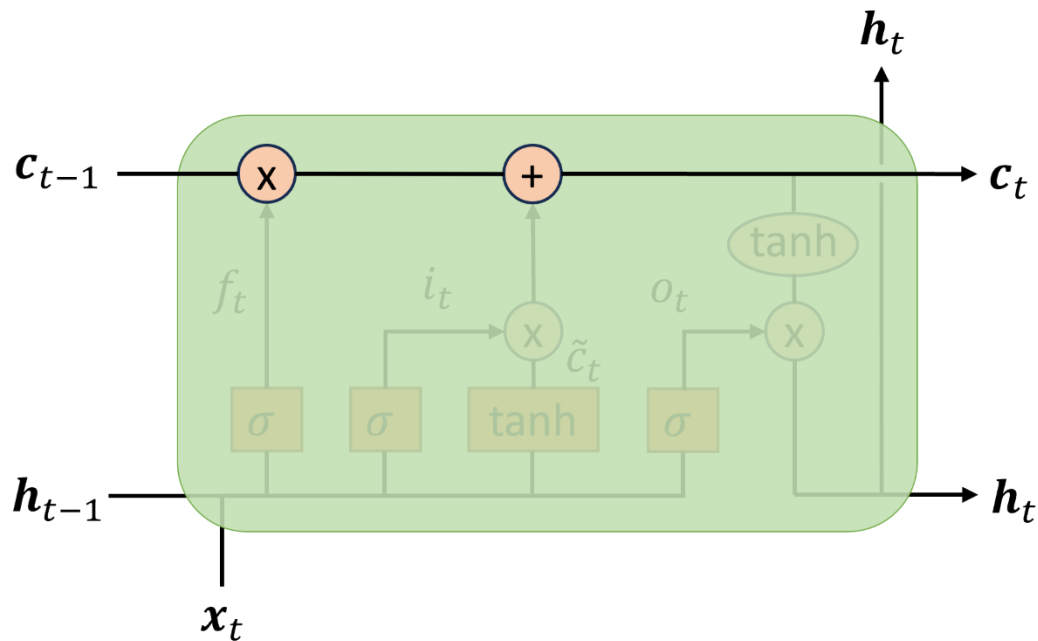
A ideia central por trás das LSTMs: estado da célula



- Os valores no vetor g_t variam entre 0 e 1, onde:
 - 0 significa “fechado”
 - e 1 significa “aberto”.
- Resumindo, a **porta “filtra” a informação** multiplicando o vetor de entrada x_t pelo vetor de porta g_t , elemento a elemento.

x_t		g_t		y_t
0.58	x	0.2	=	0.116
-0.32		0.99		-0.3168
0.01		0.1		0.001
1.24		0.95		1.178

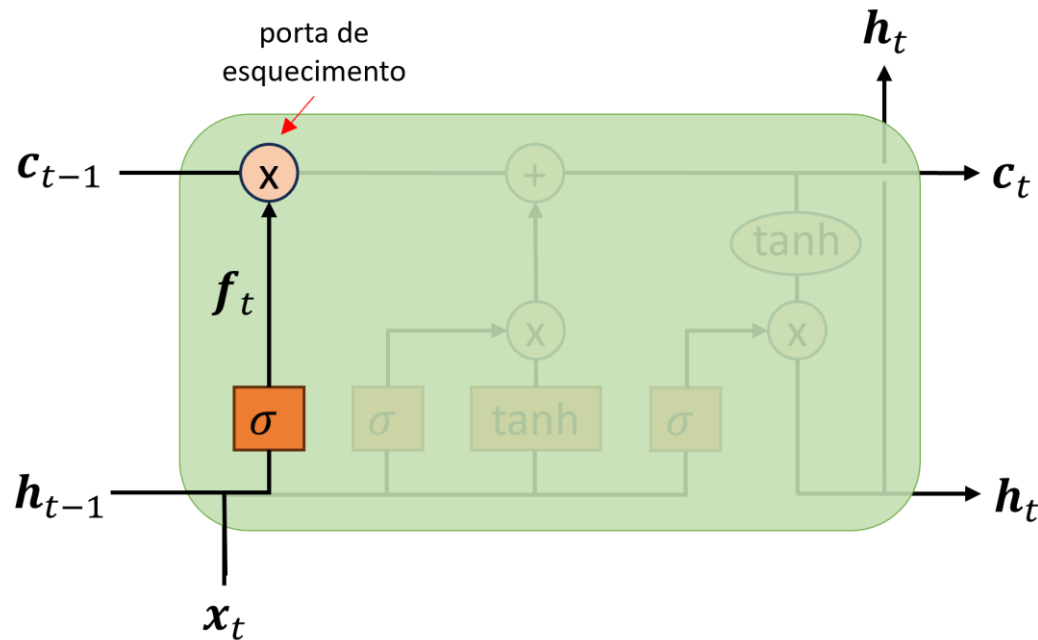
Estados da célula



- Uma célula LSTM possui dois estados:
 - o estado da célula, c_t
 - e o estado oculto da célula, h_t .
- c_t é a memória de longo prazo.
- h_t é memória de curto prazo e saída atual da célula.
- A célula modifica o estado da célula, c_t , apenas por meio de **interações lineares***:
 - A informação flui suavemente ao longo do tempo.
- A célula **decide quanto de c_t revelar** usando a **porta de saída**.

* importante, pois minimiza o desaparecimento do gradiente.

Porta de esquecimento

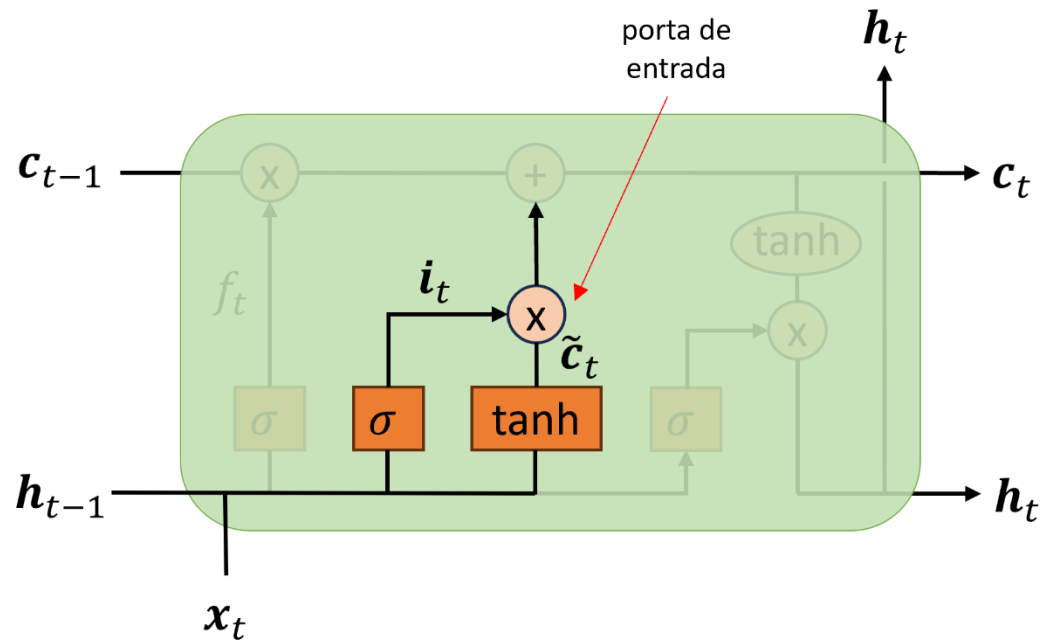


- Controla quais informações devem ser descartadas do estado de longo prazo da célula.

$$f_t = \begin{cases} 0, & \text{esquece completamente } c_{t-1} \\ 1, & \text{mantém 100\% de } c_{t-1} \end{cases}$$

- Porta controlada pela entrada atual x_t e pela saída da célula anterior h_{t-1} .
- Exemplo de PLN:
 - O estado da célula mantém o gênero do sujeito presente para usar os pronomes corretos.
 - Ao ver um novo sujeito, esqueça o gênero do sujeito anterior.

Porta de entrada



- Determina o quanto atualizar o estado antigo c_t com um novo candidato $\tilde{c}_t$.
$$i_t = \begin{cases} 0, & \text{sem atualização} \\ 1, & \text{atualiza com 100\% de } \tilde{c}_t \end{cases}$$
- No exemplo de PNL:
 - Gostaríamos de adicionar o gênero do novo sujeito ao estado da célula, para substituir o antigo que estamos esquecendo.

Atualização do estado da célula

- Calcular o vetor da porta de esquecimento

$$\mathbf{f}_t = \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t]\mathbf{W}_f + \mathbf{b}_f)$$

- Calcular o vetor da porta de entrada

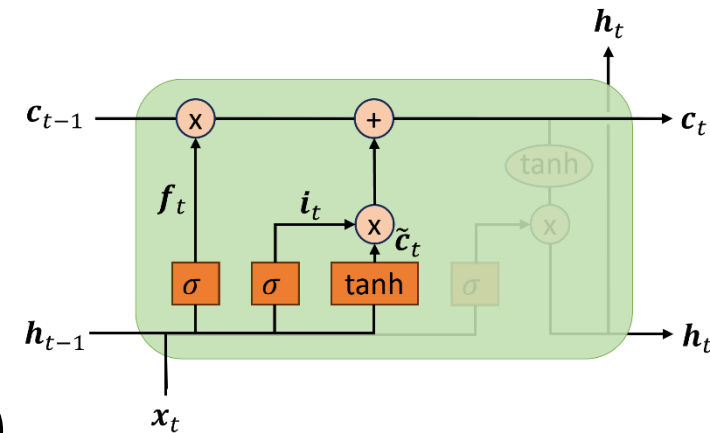
$$\mathbf{i}_t = \sigma([\mathbf{h}_{t-1}, \mathbf{x}_t]\mathbf{W}_i + \mathbf{b}_i)$$

- Calcular o vetor de informação

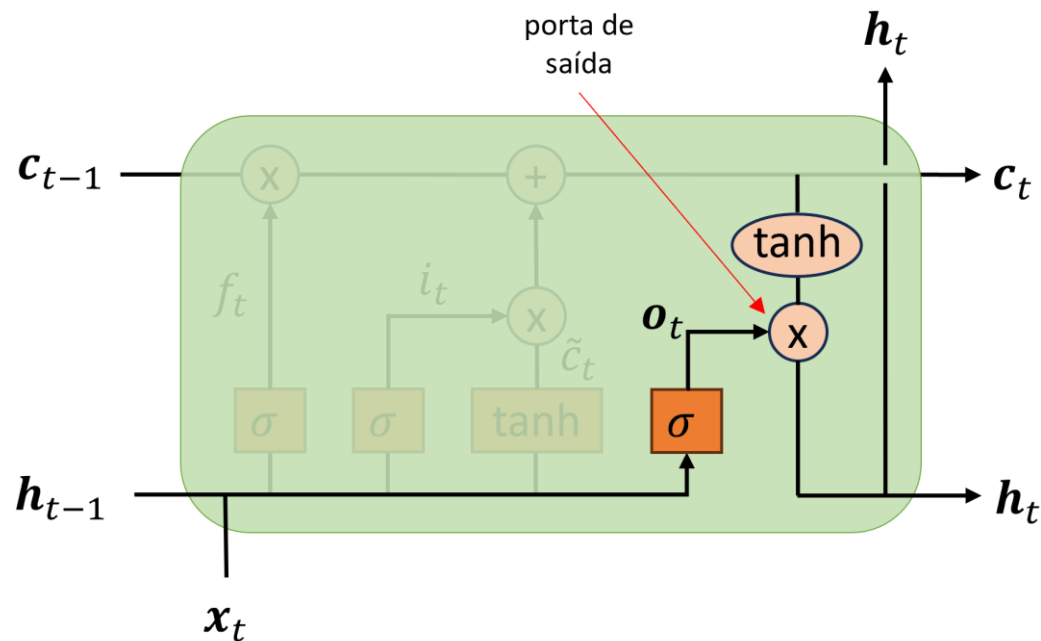
$$\tilde{\mathbf{c}}_t = \tanh([\mathbf{h}_{t-1}, \mathbf{x}_t]\mathbf{W}_c + \mathbf{b}_c)$$

- Calcular o novo vetor de estado

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \mathbf{c}_{t-1}}_{\text{esquece da informação anterior}} + \underbrace{\mathbf{i}_t \tilde{\mathbf{c}}_t}_{\text{entada de nova informação}},$$



Porta de saída



- A saída é uma versão filtrada do estado da célula, c_t .
- c_t é passado através de uma $\tanh$, que comprime seus valores entre -1 e 1.
- Em seguida, a porta decide quais partes de c_t serão passadas para a saída.
$$o_t = \begin{cases} 0, & \text{sem saída} \\ 1, & \text{passa 100\% do estado} \end{cases}$$
- No exemplo de PLN:
 - Pode indicar se o sujeito é singular ou plural, para que se um verbo vier em seguida, ele seja conjugado corretamente.

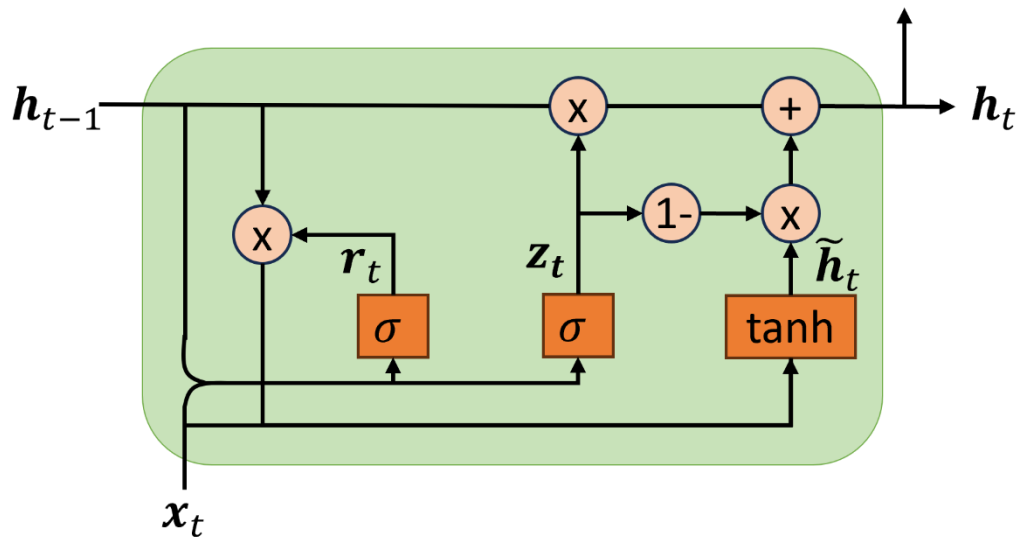
Long-short term memory (LSTM)

- Resumindo, uma célula LSTM pode:
 - aprender a reconhecer uma entrada importante (essa é a função da porta de entrada),
 - armazená-la no estado de longo prazo, C_t ,
 - preservá-la pelo tempo que for necessário (essa é a função da porta do esquecimento)
 - e extraí-la sempre que necessário (essa é a função da porta de saída).

Principais desvantagens de LSTMs

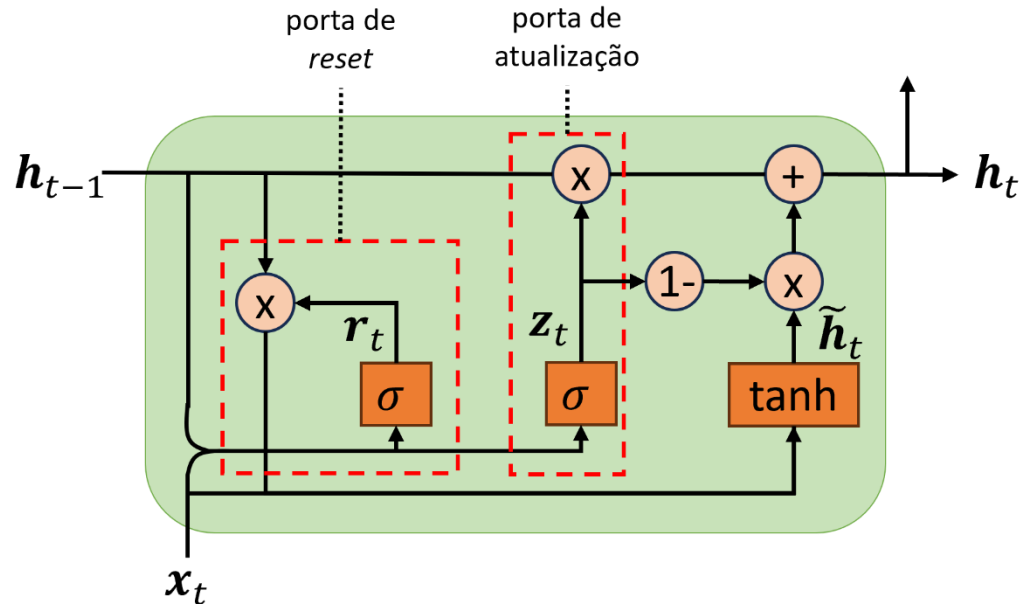
- As saídas das portas nunca são realmente 0 ou 1.
 - Assim, o conteúdo da célula inevitavelmente se corrompe após um longo período.
 - Portanto, ainda apresenta dificuldades com dependências temporais muito longas.
- Alta complexidade computacional.
 - Muito mais parâmetros que RNN simples.
- Dificuldade de paralelização
 - Processamento sequencial: depende de $t - 1$ para calcular t
 - Difícil explorar paralelismo em GPU
- Hoje é superada em muitos cenários
 - Modelos mais modernos: Transformers com attention
 - Eles apresentam melhor: paralelização e capturam de dependências globais

Gated recurrent unit (GRU)



- Com o sucesso das LSTMs em capturar dependências de longo prazo, arquiteturas simplificadas foram propostas com o objetivo de acelerar a computação.
- GRUs foram propostas em 2014 por Cho et al.
- São uma versão simplificada da célula de memória LSTM.
- Em geral, atingem desempenho comparável, mas com a vantagem de terem menos parâmetros e serem mais rápidas em termos de computação.

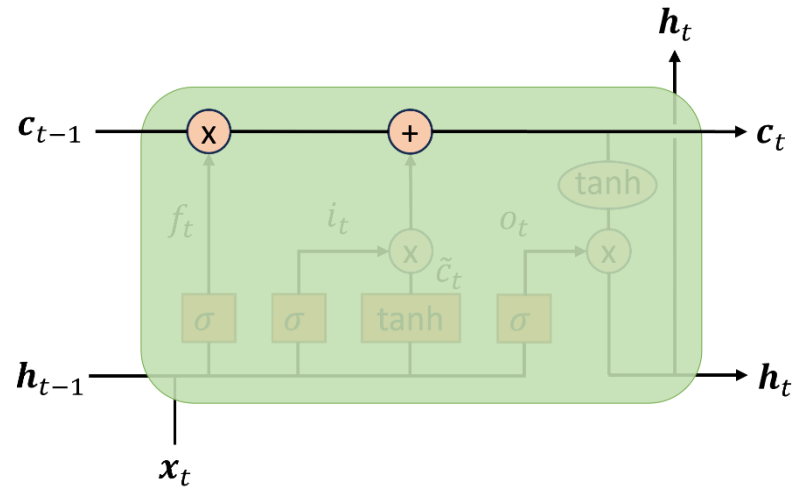
Gated recurrent unit (GRU)



- Usam menos portas e não possuem um estado da célula.
- A porta de *reset* é responsável pela memória de curto prazo, pois decide quanta informação passada será mantida e quanta informação será descartada.
- Em contraste, a porta de atualização é responsável pela memória de longo prazo e é comparável à porta de esquecimento da LSTM.
 - É a combinação das portas de esquecimento e de entrada da LSTM.

Como LSTMs e GRUs minimizam o problema do desaparecimento do gradiente?

Desaparecimento do gradiente

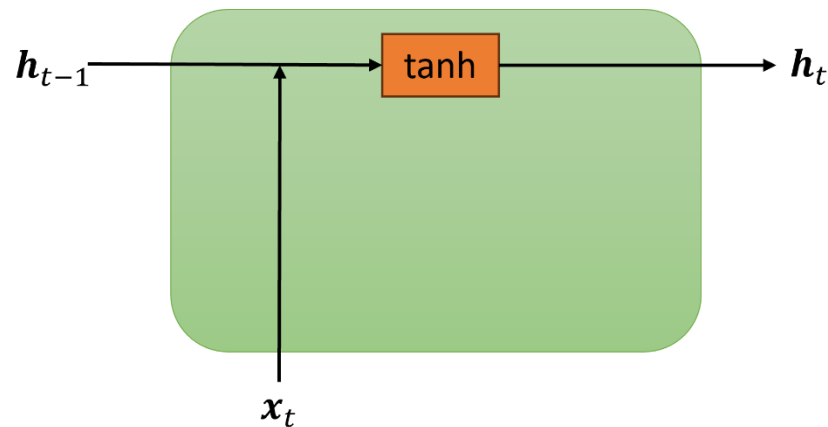


- GRUs e LSTMs têm um caminho quase linear para o gradiente.

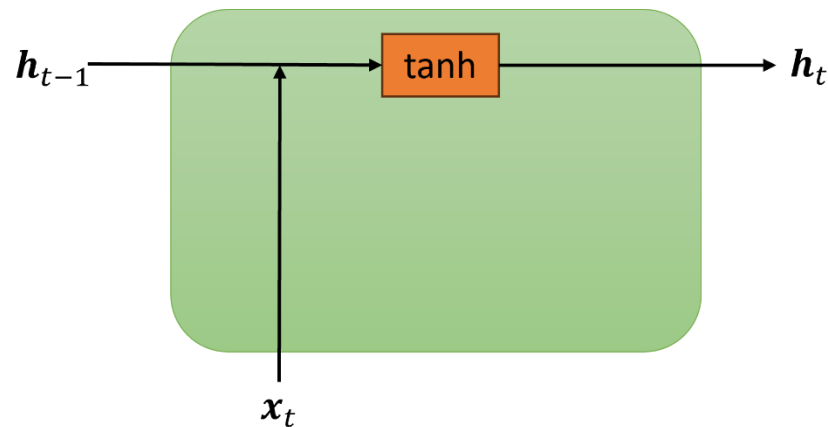
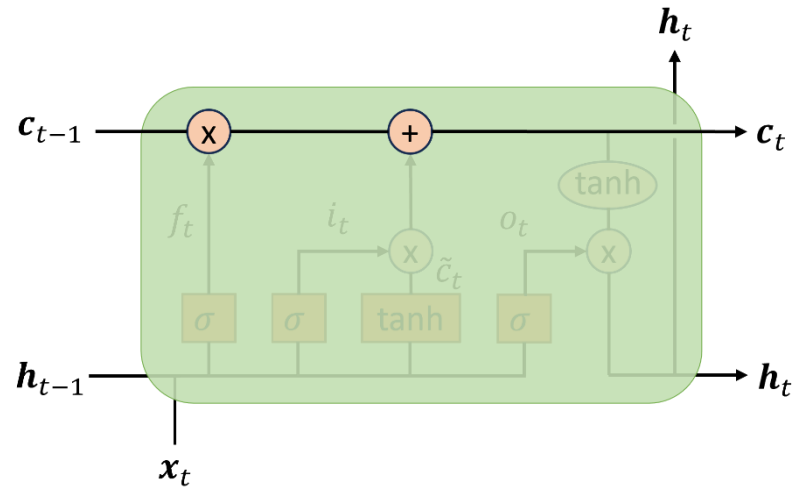
$$c_t = f_t c_{t-1} + i_t \tilde{c}_t.$$

- Já as RNNs baunilha têm uma caminho não-linear: o novo estado passa por um função tanh.

$$h_t = \tanh([x_t, h_{t-1}]).$$

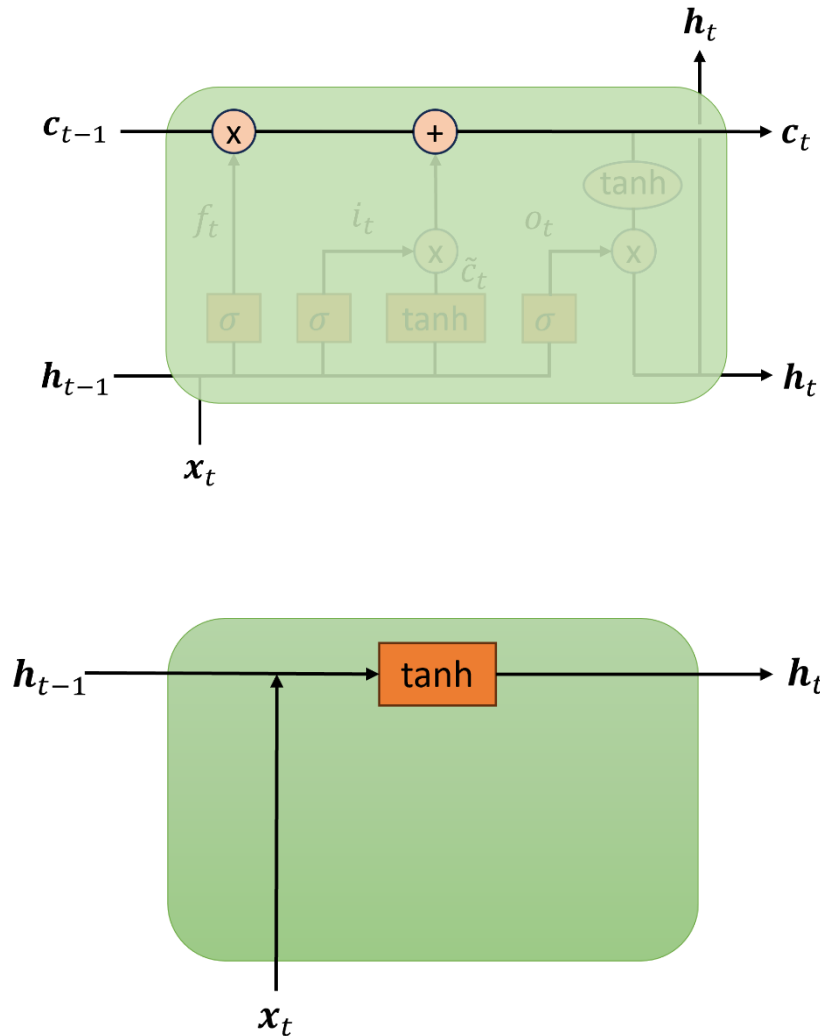


Desaparecimento do gradiente



- LSTMs e GRUs minimizam o problema do desaparecimento do gradiente porque a atualização do estado da célula é aditiva e não puramente multiplicativa.
- A soma cria um “atalho” **configurável** para o gradiente.
- Esse atalho permite que os gradientes retrocedam no tempo sem necessariamente desaparecer.

Desaparecimento do gradiente



- Para LSTMs e GRUs, o valor da derivada $\frac{\partial c_t}{\partial c_{t-1}}$ é função das 3 portas: f_t , i_t e o_t e do estado de atualização $\tilde{c}_t$.
- Esses 4 valores são configurações que a rede aprende a definir.
- Dessa forma, a rede aprende a decidir quando deixar o gradiente desaparecer e quando preservá-lo, ajustando os valores das portas.
- Assim, as portas permitem que a rede decida o que guardar e o que descartar.

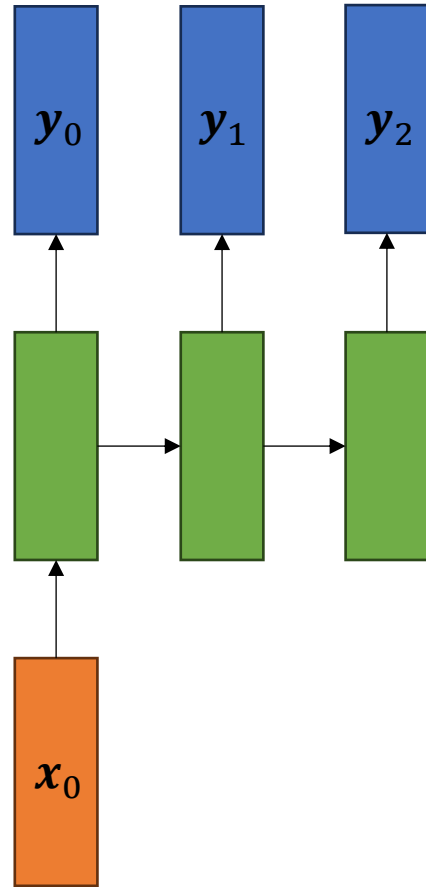
Diferentes aplicações de RNNs

Aplicação: um-para-um



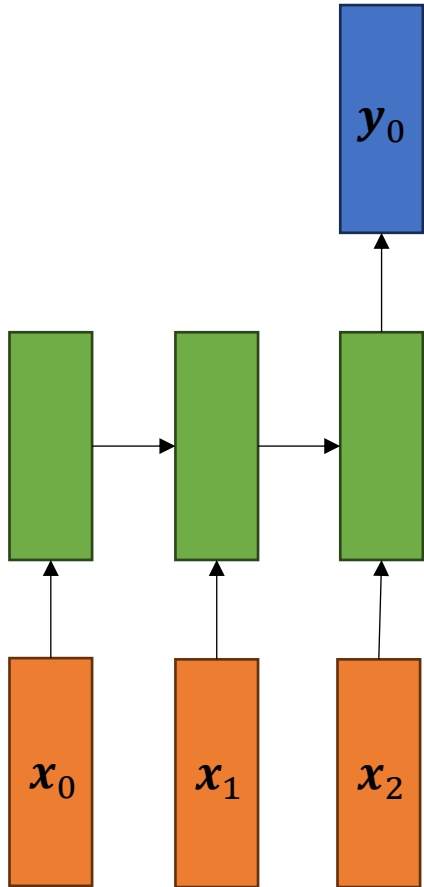
- Rede neural de alimentação direta tradicional.
- Entrada e saída de tamanhos fixos.
- Não há dados sequenciais envolvidos.
- Exemplo de uso:
 - Classificação de imagens.
 - Entrada: imagem
 - Saída: classe

Aplicação: um-para-vários



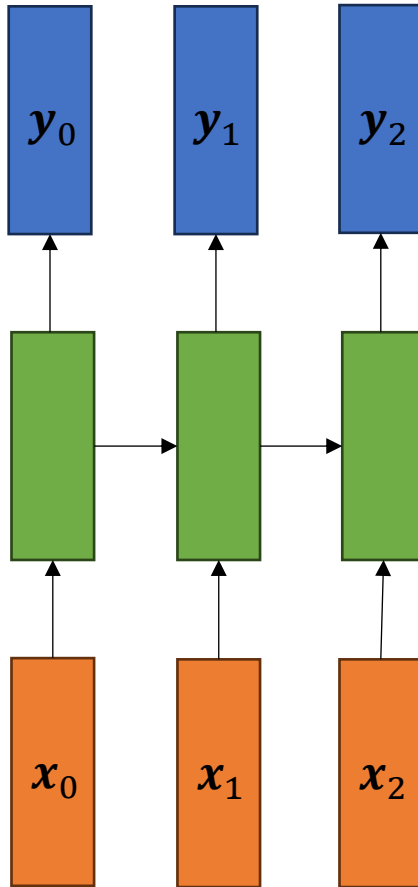
- Entrada de tamanho fixo.
- Saída de tamanho variável (sequência).
- Exemplo de uso:
 - Legendas de imagens.
 - Imagem de entrada -> sequência de palavras descrevendo a imagem na saída.
 - Geração de música.

Aplicação: vários-para-um



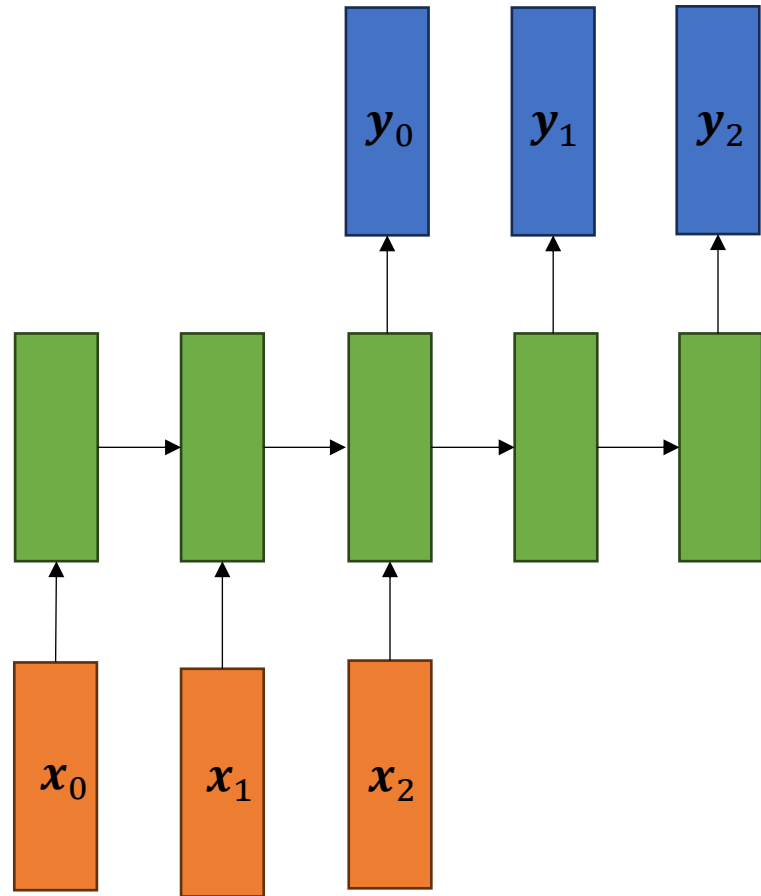
- Entrada de tamanho variável (sequência).
- Saída de tamanho fixo.
- Exemplo de uso:
 - Classificação de sentimentos.
 - Por exemplo, classificar o sentimento em uma mensagem de rede social.
 - ✓ Entrada: sequência de palavras
 - ✓ Saída: sentimento.

Aplicação: vários-para-vários síncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Entrada e saída sincronizadas.
 - Comprimentos de entrada e saída são iguais.
- Exemplos de uso:
 - Classificação de vídeo em nível de quadro.
 - Contagem de tráfego:
 - ✓ Entrada: Imagem de carros em um semáforo.
 - ✓ Saída: quantos carros na imagem.
 - Etiquetagem gramatical
 - Entrada: sequência de palavras
 - Saída: classe gramatical de cada palavra

Aplicação: vários-para-vários assíncrono



- Entrada de tamanho variável (sequência).
- Saída de tamanho variável (sequência).
- Entrada e saída não são sincronizadas.
 - Comprimentos de entrada e saída não são necessariamente iguais.
- Exemplo de uso:
 - Tradução automática e chatbots
 - Entrada: sequência de palavras.
 - Saída: sequência de palavras.

Exemplo

- [Exemplo: rnn.ipynb](#)



Lista de exercícios

[Lista #11 - RNN](#)

Perguntas?

Obrigado!

Figuras

